https://paddyspeaks.com/articles/ma-architecture-patterns.html

Performance Accessibility Best Practices SEO

0–49 50–89 90–100

There were issues affecting this run of Lighthouse:

- **The page did not paint any content. Please ensure you keep the browser window in the foreground during the load and try again. (NO_FCP)**

Performance

Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.

METRICS Collapse view

First Contentful Paint

Error!

The page did not paint any content. Please ensure you keep the browser window in the foreground during the load and try again. (NO_FCP)

Largest Contentful Paint

Error!

The page did not paint any content. Please ensure you keep the browser window in the foreground during the load and try again. (NO_FCP)

Total Blocking Time

Error!

The page did not paint any content. Please ensure you keep the browser window in the foreground during the load and try again. (NO_FCP)

Cumulative Layout Shift

Error!

The page did not paint any content. Please ensure you keep the browser window in the foreground during the load and try again. (NO_FCP)

Speed Index

Error!

The page did not paint any content. Please ensure you keep the browser window in the foreground during the load and try again. (NO_FCP)

INSIGHTS

! Use efficient cache lifetimes — Error!

A long cache lifetime can speed up repeat visits to your page. Learn more about caching. Unscored

! Layout shift culprits — Error!

Layout shifts occur when elements move absent any user interaction. Investigate the causes of layout shifts, such as elements being added, removed, or their fonts changing as the page loads. Unscored

! Document request latency — Error!

Your first network request is the most important. Reduce its latency by avoiding redirects, ensuring a fast server response, and enabling text compression. Unscored

! Optimize DOM size — Error!

A large DOM can increase the duration of style calculations and layout reflows, impacting page responsiveness. A large DOM will also increase memory usage. Learn how to avoid an excessive DOM size. Unscored

! Font display — Error!

Consider setting `font-display` to `swap` or `optional` to ensure text is consistently visible. `swap` can be further optimized to mitigate layout shifts with font metric overrides. Unscored

! Forced reflow — Error!

A forced reflow occurs when JavaScript queries geometric properties (such as `offsetWidth`) after styles have been invalidated by a change to the DOM state. This can result in poor performance. Learn more about forced reflows and possible mitigations. Unscored

! Improve image delivery — Error!

Reducing the download time of images can improve the perceived load time of the page and LCP. Learn more about optimizing image size Unscored

! INP breakdown — Error!

Start investigating how to improve INP by looking at the longest subpart. Unscored

! LCP breakdown — Error!

Each subpart has specific improvement strategies. Ideally, most of the LCP time should be spent on loading the resources, not within delays. Unscored

! LCP request discovery — Error!

Optimize LCP by making the LCP image discoverable from the HTML immediately, and avoiding lazy-loading Unscored

! Modern HTTP — Error!

HTTP/2 and HTTP/3 offer many benefits over HTTP/1.1, such as multiplexing. Learn more about using modern HTTP. Unscored

! Render-blocking requests — Error!

Requests are blocking the page's initial render, which may delay LCP. Deferring or inlining can move these network requests out of the critical path. Unscored

! 3rd parties — Error!

3rd party code can significantly impact load performance. Reduce and defer loading of 3rd party code to prioritize your page's content. Unscored

! Optimize viewport for mobile — Error!

Tap interactions may be delayed by up to 300 ms if the viewport is not optimized for mobile. Unscored

! Duplicated JavaScript — Error!

Remove large, duplicate JavaScript modules from bundles to reduce unnecessary bytes consumed by network activity. Unscored

! Legacy JavaScript — Error!

Polyfills and transforms enable older browsers to use new JavaScript features. However, many aren't necessary for modern browsers. Consider modifying your JavaScript build process to not transpile Baseline features, unless you know you must support older browsers. Learn why most sites can deploy ES6+ code without transpiling Unscored

! Network dependency tree — Error!

Avoid chaining critical requests by reducing the length of chains, reducing the download size of resources, or deferring the download of unnecessary resources to improve page load. Unscored

These insights are also available in the Chrome DevTools Performance Panel - record a trace to view more detailed information.

DIAGNOSTICS

! Image elements have explicit `width` and `height` — Error!

Set an explicit width and height on image elements to reduce layout shifts and improve CLS. Learn how to set image dimensions Unscored

! Page didn't prevent back/forward cache restoration — Error!

Many navigations are performed by going back to a previous page, or forwards again. The back/forward cache (bfcache) can speed up these return navigations. Learn more about the bfcache Unscored

! Minify CSS — Error!

Minifying CSS files can reduce network payload sizes. Learn how to minify CSS. Unscored

! Minify JavaScript — Error!

Minifying JavaScript files can reduce payload sizes and script parse time. Learn how to minify JavaScript. Unscored

! User Timing marks and measures — Error!

Consider instrumenting your app with the User Timing API to measure your app's real-world performance during key user experiences. Learn more about User Timing marks. Unscored

! Avoid non-composited animations — Error!

Animations which are not composited can be janky and increase CLS. Learn how to avoid non-composited animations Unscored

! Reduce unused CSS — Error!

Reduce unused rules from stylesheets and defer CSS not used for above-the-fold content to decrease bytes consumed by network activity. Learn how to reduce unused CSS. Unscored

! Reduce unused JavaScript — Error!

Reduce unused JavaScript and defer loading scripts until they are required to decrease bytes consumed by network activity. Learn how to reduce unused JavaScript. Unscored

! Avoids enormous network payloads — Error!

Large network payloads cost users real money and are highly correlated with long load times. Learn how to reduce payload sizes. Unscored

! JavaScript execution time — Error!

Consider reducing the time spent parsing, compiling, and executing JS. You may find delivering smaller JS payloads helps with this. Learn how to reduce Javascript execution time. Unscored

! Minimizes main-thread work — Error!

Consider reducing the time spent parsing, compiling and executing JS. You may find delivering smaller JS payloads helps with this. Learn how to minimize main-thread work Unscored

! Avoid long main-thread tasks — Error!

Lists the longest tasks on the main thread, useful for identifying worst contributors to input delay. Learn how to avoid long main-thread tasks Unscored

More information about the performance of your application. These numbers don't directly affect the Performance score.

Accessibility

These checks highlight opportunities to improve the accessibility of your web app. Automatic detection can only detect a subset of issues and does not guarantee the accessibility of your web app, so manual testing is also encouraged.

! Interactive controls are keyboard focusable — Error!

Custom interactive controls are keyboard focusable and display a focus indicator. Learn how to make custom controls focusable. Unscored

! Interactive elements indicate their purpose and state — Error!

Interactive elements, such as links and buttons, should indicate their state and be distinguishable from non-interactive elements. Learn how to decorate interactive elements with affordance hints. Unscored

! The page has a logical tab order — Error!

Tabbing through the page follows the visual layout. Users cannot focus elements that are offscreen. Learn more about logical tab ordering. Unscored

! Visual order on the page follows DOM order — Error!

DOM order matches the visual order, improving navigation for assistive technology. Learn more about DOM and visual ordering. Unscored

! User focus is not accidentally trapped in a region — Error!

A user can tab into and out of any control or region without accidentally trapping their focus. Learn how to avoid focus traps. Unscored

! The user's focus is directed to new content added to the page — Error!

If new content, such as a dialog, is added to the page, the user's focus is directed to it. Learn how to direct focus to new content. Unscored

! HTML5 landmark elements are used to improve navigation — Error!

Landmark elements (`<main>`, `<nav>`, etc.) are used to improve the keyboard navigation of the page for assistive technology. Learn more about landmark elements. Unscored

! Offscreen content is hidden from assistive technology — Error!

Offscreen content is hidden with display: none or aria-hidden=true. Learn how to properly hide offscreen content. Unscored

! Custom controls have associated labels — Error!

Custom interactive controls have associated labels, provided by aria-label or aria-labelledby. Learn more about custom controls and labels. Unscored

! Custom controls have ARIA roles — Error!

Custom interactive controls have appropriate ARIA roles. Learn how to add roles to custom controls. Unscored

ARIA

! `[aria-*]` attributes match their roles — Error!

Each ARIA `role` supports a specific subset of `aria-*` attributes. Mismatching these invalidates the `aria-*` attributes. Learn how to match ARIA attributes to their roles.

! `[aria-hidden="true"]` is not present on the document `<body>` — Error!

Assistive technologies, like screen readers, work inconsistently when `aria-hidden="true"` is set on the document `<body>`. Learn how `aria-hidden` affects the document body.

! `[role]`s have all required `[aria-*]` attributes — Error!

Some ARIA roles have required attributes that describe the state of the element to screen readers. Learn more about roles and required attributes.

! Elements with an ARIA `[role]` that require children to contain a specific `[role]` have all required children. — Error!

Some ARIA parent roles must contain specific child roles to perform their intended accessibility functions. Learn more about roles and required children elements.

! `[role]`s are contained by their required parent element — Error!

Some ARIA child roles must be contained by specific parent roles to properly perform their intended accessibility functions. Learn more about ARIA roles and required parent element.

! `[role]` values are valid — Error!

ARIA roles must have valid values in order to perform their intended accessibility functions. Learn more about valid ARIA roles.

! `[aria-*]` attributes have valid values — Error!

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid values. Learn more about valid values for ARIA attributes.

! `[aria-*]` attributes are valid and not misspelled — Error!

Assistive technologies, like screen readers, can't interpret ARIA attributes with invalid names. Learn more about valid ARIA attributes.

! ARIA IDs are unique — Error!

The value of an ARIA ID must be unique to prevent other instances from being overlooked by assistive technologies. Learn how to fix duplicate ARIA IDs.

! `button`, `link`, and `menuitem` elements have accessible names — Error!

When an element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to make command elements more accessible.

! ARIA attributes are used as specified for the element's role — Error!

Some ARIA attributes are only allowed on an element under certain conditions. Learn more about conditional ARIA attributes.

! Elements with `role="dialog"` or `role="alertdialog"` have accessible names. — Error!

ARIA dialog elements without accessible names may prevent screen readers users from discerning the purpose of these elements. Learn how to make ARIA dialog elements more accessible.

! `[aria-hidden="true"]` elements do not contain focusable descendents — Error!

Focusable descendents within an `[aria-hidden="true"]` element prevent those interactive elements from being available to users of assistive technologies like screen readers. Learn how `aria-hidden` affects focusable elements.

! ARIA input fields have accessible names — Error!

When an input field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about input field labels.

! ARIA `meter` elements have accessible names — Error!

When a meter element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `meter` elements.

! ARIA `progressbar` elements have accessible names — Error!

When a `progressbar` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to label `progressbar` elements.

! Elements use only permitted ARIA attributes — Error!

Using ARIA attributes in roles where they are prohibited can mean that important information is not communicated to users of assistive technologies. Learn more about prohibited ARIA roles.

! Elements with the `role=text` attribute do not have focusable descendents. — Error!

Adding `role=text` around a text node split by markup enables VoiceOver to treat it as one phrase, but the element's focusable descendents will not be announced. Learn more about the `role=text` attribute.

! ARIA toggle fields have accessible names — Error!

When a toggle field doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about toggle fields.

! ARIA `tooltip` elements have accessible names — Error!

When a tooltip element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn how to name `tooltip` elements.

! ARIA `treeitem` elements have accessible names — Error!

When a `treeitem` element doesn't have an accessible name, screen readers announce it with a generic name, making it unusable for users who rely on screen readers. Learn more about labeling `treeitem` elements.

! Deprecated ARIA roles were not used — Error!

Deprecated ARIA roles may not be processed correctly by assistive technology. Learn more about deprecated ARIA roles.

These are opportunities to improve the usage of ARIA in your application which may enhance the experience for users of assistive technology, like a screen reader.

NAMES AND LABELS

! Buttons have an accessible name — Error!

When a button doesn't have an accessible name, screen readers announce it as "button", making it unusable for users who rely on screen readers. Learn how to make buttons more accessible.

! Image elements have `[alt]` attributes — Error!

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute.

! Input buttons have discernible text. — Error!

Adding discernable and accessible text to input buttons may help screen reader users understand the purpose of the input button. Learn more about input buttons.

! `<input type="image">` elements have `[alt]` text — Error!

When an image is being used as an `<input>` button, providing alternative text can help screen reader users understand the purpose of the button. Learn about input image alt text.

! Form elements have associated labels — Error!

Labels ensure that form controls are announced properly by assistive technologies, like screen readers. Learn more about form element labels.

! Select elements have associated label elements. — Error!

Form elements without effective labels can create frustrating experiences for screen reader users. Learn more about the `select` element.

! Document has a `<title>` element — Error!

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

! `<frame>` or `<iframe>` elements have a title — Error!

Screen reader users rely on frame titles to describe the contents of frames. Learn more about frame titles.

! Links have a discernible name — Error!

Link text (and alternate text for images, when used as links) that is discernible, unique, and focusable improves the navigation experience for screen reader users. Learn how to make links accessible.

! `<object>` elements have alternate text — Error!

Screen readers cannot translate non-text content. Adding alternate text to `<object>` elements helps screen readers convey meaning to users. Learn more about alt text for `object` elements.

! No form fields have multiple labels — Error!

Form fields with multiple labels can be confusingly announced by assistive technologies like screen readers which use either the first, the last, or all of the labels. Learn how to use form labels.

! Skip links are focusable. — Error!

Including a skip link can help users skip to the main content to save time. Learn more about skip links.

! Image elements do not have `[alt]` attributes that are redundant text. — Error!

Informative elements should aim for short, descriptive alternative text. Alternative text that is exactly the same as the text adjacent to the link or image is potentially confusing for screen reader users, because the text will be read twice. Learn more about the `alt` attribute. Unscored

These are opportunities to improve the semantics of the controls in your application. This may enhance the experience for users of assistive technology, like a screen reader.

BEST PRACTICES

! The document does not use `<meta http-equiv="refresh">` — Error!

Users do not expect a page to refresh automatically, and doing so will move focus back to the top of the page. This may create a frustrating or confusing experience. Learn more about the refresh meta tag.

! `[user-scalable="no"]` is not used in the `<meta name="viewport">` element and the `[maximum-scale]` attribute is not less than 5. — Error!

Disabling zooming is problematic for users with low vision who rely on screen magnification to properly see the contents of a web page. Learn more about the viewport meta tag.

! Touch targets have sufficient size and spacing. — Error!

Touch targets with sufficient size and spacing help users who may have difficulty targeting small controls to activate the targets. Learn more about touch targets.

! Document has a main landmark. — Error!

One main landmark helps screen reader users navigate a web page. Learn more about landmarks.

! Tables have different content in the summary attribute and `<caption>`. — Error!

The summary attribute should describe the table structure, while `<caption>` should have the onscreen title. Accurate table mark-up helps users of screen readers. Learn more about summary and caption. Unscored

! All heading elements contain content. — Error!

A heading with no content or inaccessible text prevent screen reader users from accessing information on the page's structure. Learn more about headings. Unscored

! Uses ARIA roles only on compatible elements — Error!

Many HTML elements can only be assigned certain ARIA roles. Using ARIA roles where they are not allowed can interfere with the accessibility of the web page. Learn more about ARIA roles. Unscored

! Identical links have the same purpose. — Error!

Links with the same destination should have the same description, to help users understand the link's purpose and decide whether to follow it. Learn more about identical links. Unscored

These items highlight common accessibility best practices.

AUDIO AND VIDEO

! `<video>` elements contain a `<track>` element with `[kind="captions"]` — Error!

When a video provides a caption it is easier for deaf and hearing impaired users to access its information. Learn more about video captions.

These are opportunities to provide alternative content for audio and video. This may improve the experience for users with hearing or vision impairments.

NAVIGATION

! `[accesskey]` values are unique — Error!

Access keys let users quickly focus a part of the page. For proper navigation, each access key must be unique. Learn more about access keys.

! The page contains a heading, skip link, or landmark region — Error!

Adding ways to bypass repetitive content lets keyboard users navigate the page more efficiently. Learn more about bypass blocks.

! No element has a `[tabindex]` value greater than 0 — Error!

A value greater than 0 implies an explicit navigation ordering. Although technically valid, this often creates frustrating experiences for users who rely on assistive technologies. Learn more about the `tabindex` attribute.

! Heading elements appear in a sequentially-descending order — Error!

Properly ordered headings that do not skip levels convey the semantic structure of the page, making it easier to navigate and understand when using assistive technologies. Learn more about heading order.

These are opportunities to improve keyboard navigation in your application.

CONTRAST

! Background and foreground colors have a sufficient contrast ratio — Error!

Low-contrast text is difficult or impossible for many users to read. Learn how to provide sufficient color contrast.

! Links are distinguishable without relying on color. — Error!

Low-contrast text is difficult or impossible for many users to read. Link text that is discernible improves the experience for users with low vision. Learn how to make links distinguishable.

These are opportunities to improve the legibility of your content.

TABLES AND LISTS

! `<dl>`'s contain only properly-ordered `<dt>` and `<dd>` groups, `<script>`, `<template>` or `<div>` elements. — Error!

When definition lists are not properly marked up, screen readers may produce confusing or inaccurate output. Learn how to structure definition lists correctly.

! Definition list items are wrapped in `<dl>` elements — Error!

Definition list items (`<dt>` and `<dd>`) must be wrapped in a parent `<dl>` element to ensure that screen readers can properly announce them. Learn how to structure definition lists correctly.

! Lists contain only `<li>` elements and script supporting elements (`<script>` and `<template>`). — Error!

Screen readers have a specific way of announcing lists. Ensuring proper list structure aids screen reader output. Learn more about proper list structure.

! List items (`<li>`) are contained within `<ul>`, `<ol>` or `<menu>` parent elements — Error!

Screen readers require list items (`<li>`) to be contained within a parent `<ul>`, `<ol>` or `<menu>` to be announced properly. Learn more about proper list structure.

! Cells in a `<table>` element that use the `[headers]` attribute refer to table cells within the same table. — Error!

Screen readers have features to make navigating tables easier. Ensuring `<td>` cells using the `[headers]` attribute only refer to other cells in the same table may improve the experience for screen reader users. Learn more about the `headers` attribute.

! `<th>` elements and elements with `[role="columnheader"/"rowheader"]` have data cells they describe. — Error!

Screen readers have features to make navigating tables easier. Ensuring table headers always refer to some set of cells may improve the experience for screen reader users. Learn more about table headers.

These are opportunities to improve the experience of reading tabular or list data using assistive technology, like a screen reader.

INTERNATIONALIZATION AND LOCALIZATION

! `<html>` element has a `[lang]` attribute — Error!

If a page doesn't specify a `lang` attribute, a screen reader assumes that the page is in the default language that the user chose when setting up the screen reader. If the page isn't actually in the default language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

! `<html>` element has a valid value for its `[lang]` attribute — Error!

Specifying a valid BCP 47 language helps screen readers announce text properly. Learn how to use the `lang` attribute.

! `[lang]` attributes have a valid value — Error!

Specifying a valid BCP 47 language on elements helps ensure that text is pronounced correctly by a screen reader. Learn how to use the `lang` attribute.

! `<html>` element has an `[xml:lang]` attribute with the same base language as the `[lang]` attribute. — Error!

If the webpage does not specify a consistent language, then the screen reader might not announce the page's text correctly. Learn more about the `lang` attribute.

These are opportunities to improve the interpretation of your content by users in different locales.

Best Practices

TRUST AND SAFETY

! Uses HTTPS — Error!

All sites should be protected with HTTPS, even ones that don't handle sensitive data. This includes avoiding mixed content, where some resources are loaded over HTTP despite the initial request being served over HTTPS. HTTPS prevents intruders from tampering with or passively listening in on the communications between your app and your users, and is a prerequisite for HTTP/2 and many new web platform APIs. Learn more about HTTPS.

! Redirects HTTP traffic to HTTPS — Error!

Make sure that you redirect all HTTP traffic to HTTPS in order to enable secure web features for all your users. Learn more.

! Avoids requesting the geolocation permission on page load — Error!

Users are mistrustful of or confused by sites that request their location without context. Consider tying the request to a user action instead. Learn more about the geolocation permission.

! Avoids requesting the notification permission on page load — Error!

Users are mistrustful of or confused by sites that request to send notifications without context. Consider tying the request to user gestures instead. Learn more about responsibly getting permission for notifications.

! Ensure CSP is effective against XSS attacks — Error!

A strong Content Security Policy (CSP) significantly reduces the risk of cross-site scripting (XSS) attacks. Learn how to use a CSP to prevent XSS Unscored

! Use a strong HSTS policy — Error!

Deployment of the HSTS header significantly reduces the risk of downgrading HTTP connections and eavesdropping attacks. A rollout in stages, starting with a low max-age is recommended. Learn more about using a strong HSTS policy. Unscored

! Ensure proper origin isolation with COOP — Error!

The Cross-Origin-Opener-Policy (COOP) can be used to isolate the top-level window from other documents such as pop-ups. Learn more about deploying the COOP header. Unscored

! Mitigate clickjacking with XFO or CSP — Error!

The `X-Frame-Options` (XFO) header or the `frame-ancestors` directive in the `Content-Security-Policy` (CSP) header control where a page can be embedded. These can mitigate clickjacking attacks by blocking some or all sites from embedding the page. Learn more about mitigating clickjacking. Unscored

! Mitigate DOM-based XSS with Trusted Types — Error!

The `require-trusted-types-for` directive in the `Content-Security-Policy` (CSP) header instructs user agents to control the data passed to DOM XSS sink functions. Learn more about mitigating DOM-based XSS with Trusted Types. Unscored

GENERAL

! Avoids deprecated APIs — Error!

Deprecated APIs will eventually be removed from the browser. Learn more about deprecated APIs.

! Avoids third-party cookies — Error!

Third-party cookies may be blocked in some contexts. Learn more about preparing for third-party cookie restrictions.

! No browser errors logged to the console — Error!

Errors logged to the console indicate unresolved problems. They can come from network request failures and other browser concerns. Learn more about this errors in console diagnostic audit

! No issues in the `Issues` panel in Chrome Devtools — Error!

Issues logged to the `Issues` panel in Chrome Devtools indicate unresolved problems. They can come from network request failures, insufficient security controls, and other browser concerns. Open up the Issues panel in Chrome DevTools for more details on each issue.

! Detected JavaScript libraries — Error!

All front-end JavaScript libraries detected on the page. Learn more about this JavaScript library detection diagnostic audit. Unscored

! Page has valid source maps — Error!

Source maps translate minified code to the original source code. This helps developers debug in production. In addition, Lighthouse is able to provide further insights. Consider deploying source maps to take advantage of these benefits. Learn more about source maps. Unscored

USER EXPERIENCE

! Allows users to paste into input fields — Error!

Preventing input pasting is a bad practice for the UX, and weakens security by blocking password managers.Learn more about user-friendly input fields.

! Displays images with correct aspect ratio — Error!

Image display dimensions should match natural aspect ratio. Learn more about image aspect ratio.

! Serves images with appropriate resolution — Error!

Image natural dimensions should be proportional to the display size and the pixel ratio to maximize image clarity. Learn how to provide responsive images.

BROWSER COMPATIBILITY

! Page has the HTML doctype — Error!

Specifying a doctype prevents the browser from switching to quirks-mode. Learn more about the doctype declaration.

! Properly defines charset — Error!

A character encoding declaration is required. It can be done with a `<meta>` tag in the first 1024 bytes of the HTML or in the Content-Type HTTP response header. Learn more about declaring the character encoding.

SEO

These checks ensure that your page is following basic search engine optimization advice. There are many additional factors Lighthouse does not score here that may affect your search ranking, including performance on Core Web Vitals. Learn more about Google Search Essentials.

! Structured data is valid — Error!

Run the Structured Data Testing Tool to validate structured data. Learn more about Structured Data. Unscored

CRAWLING AND INDEXING

! Page isn't blocked from indexing — Error!

Search engines are unable to include your pages in search results if they don't have permission to crawl them. Learn more about crawler directives.

! Page has successful HTTP status code — Error!

Pages with unsuccessful HTTP status codes may not be indexed properly. Learn more about HTTP status codes.

! Links are crawlable — Error!

Search engines may use `href` attributes on links to crawl websites. Ensure that the `href` attribute of anchor elements links to an appropriate destination, so more pages of the site can be discovered. Learn how to make links crawlable

! robots.txt is valid — Error!

If your robots.txt file is malformed, crawlers may not be able to understand how you want your website to be crawled or indexed. Learn more about robots.txt.

To appear in search results, crawlers need access to your app.

CONTENT BEST PRACTICES

! Document has a `<title>` element — Error!

The title gives screen reader users an overview of the page, and search engine users rely on it heavily to determine if a page is relevant to their search. Learn more about document titles.

! Document has a meta description — Error!

Meta descriptions may be included in search results to concisely summarize page content. Learn more about the meta description.

! Links have descriptive text — Error!

Descriptive link text helps search engines understand your content. Learn how to make links more accessible.

! Image elements have `[alt]` attributes — Error!

Informative elements should aim for short, descriptive alternate text. Decorative elements can be ignored with an empty alt attribute. Learn more about the `alt` attribute.

! Document has a valid `hreflang` — Error!

hreflang links tell search engines what version of a page they should list in search results for a given language or region. Learn more about `hreflang`.

! Document has a valid `rel=canonical` — Error!

Canonical links suggest which URL to show in search results. Learn more about canonical links.

Format your HTML in a way that enables crawlers to better understand your app's content.

Captured at Apr 15, 2026, 1:05 PM PDT
Initial page load

Emulated Moto G Power with Lighthouse 13.0.2
Slow 4G throttling

Single page session
Using Chromium 147.0.0.0 with devtools

Generated by **Lighthouse** 13.0.2 | File an issue